5 Securing Spring

This chapter covers

- Autoconfiguring Spring Security
- Defining custom user storage
- Customizing the login page
- Securing against CSRF attacks
- Knowing your user

Have you ever noticed that most people in television sitcoms don't lock their doors? In the days of *Leave It to Beaver*, it wasn't so unusual for people to leave their doors unlocked. But it seems crazy that at a time when we're concerned with privacy and security, we see television characters enabling unhindered access to their apartments and homes.

Information is probably the most valuable item we now have; crooks are looking for ways to steal our data and identities by sneaking into unsecured applications. As software developers, we must take steps to protect the information that resides in our applications. Whether it's an email account protected with a username-password pair or a brokerage account protected with a trading PIN, security is a crucial aspect of most applications.

5.1 Enabling Spring Security

The very first step in securing your Spring application is to add the Spring Boot security starter dependency to your build. In the project's pom.xml file, add the following `<dependency>` entry:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

If you're using Spring Tool Suite, this is even easier. Right-click on the pom.xml file and select Add Starters from the Spring context menu. In the starter dependencies dialog

box, select the Spring Security entry under the Security category, as shown in figure 5.1.

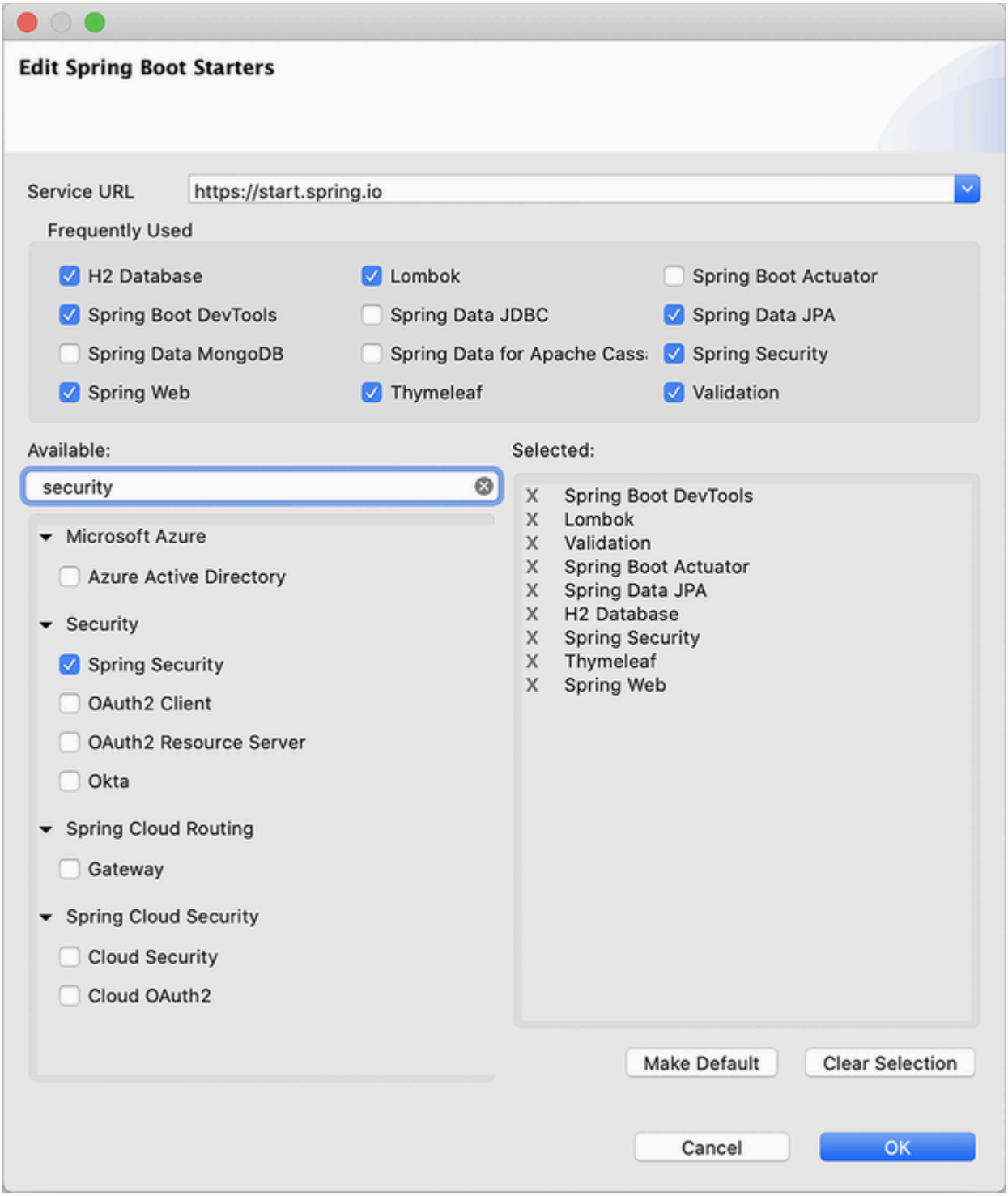


Figure 5.1 Adding the security starter with Spring Tool Suite

Believe it or not, that dependency is the only thing that's required to secure an application. When the application starts, autoconfiguration will detect that Spring Security is in the classpath and will set up some basic security configuration.

If you want to try it out, fire up the application and try to visit the home page (or any page, for that matter). You'll be prompted for authentication with a rather plain login page that looks something like figure 5.2.

TIP Going incognito: You may find it useful to set your browser to private or incognito mode when manually testing security. This will ensure that you have a fresh session each time you open a private/incognito window. You'll have to sign in to the application each time, but you can be assured that any changes you've made in security are applied and that there aren't any remnants of an older session preventing you from seeing your changes.

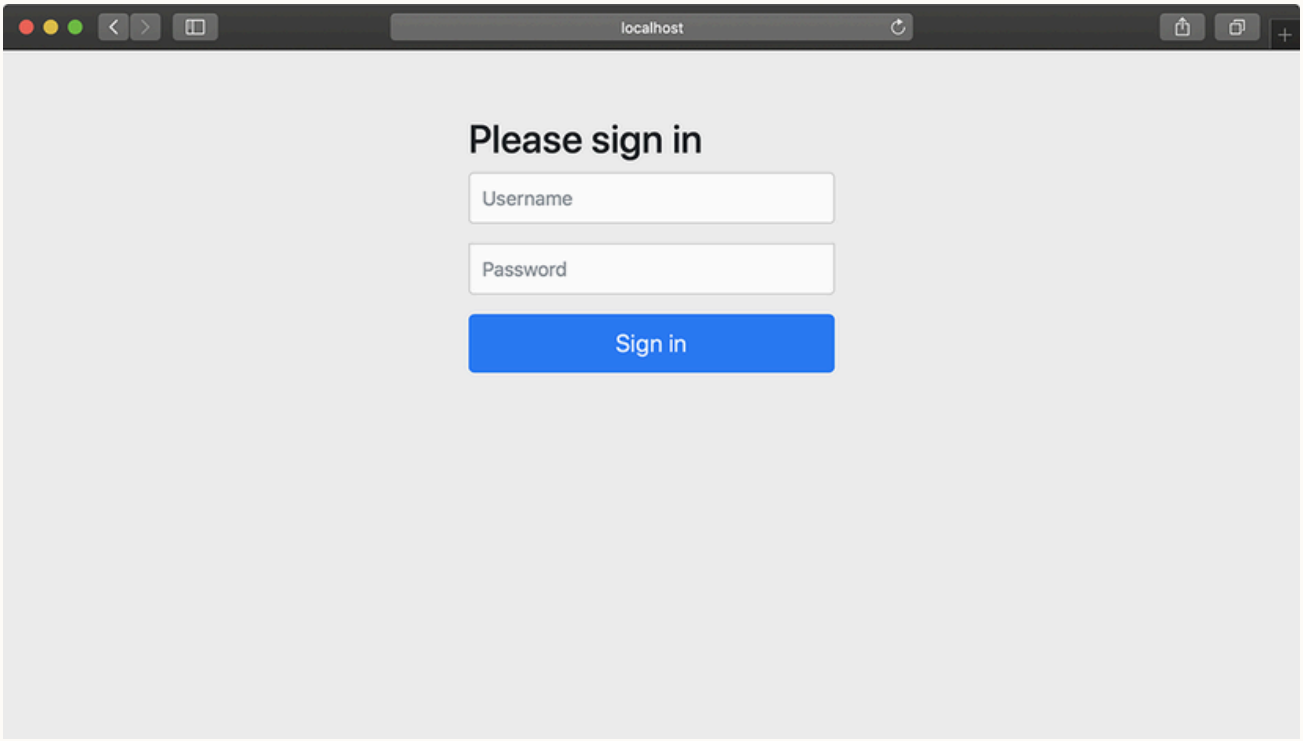


Figure 5.2 Spring Security gives you a plain login page for free.

To get past the login page, you'll need to provide a username and password. The username is *user*. As for the password, it's randomly generated and written to the application log file. The log entry will look something like this:

```
Using generated security password: 087cfc6a-027d-44bc-95d7-cbb3a798a1ea
```

Assuming you enter the username and password correctly, you'll be granted access to the application.

It seems that securing Spring applications is pretty easy work. With the Taco Cloud application secured, I suppose I could end this chapter now and move on to the next topic. But before we get ahead of ourselves, let's consider what kind of security autoconfiguration has provided.

By doing nothing more than adding the security starter to the project build, you get the following security features:

- All HTTP request paths require authentication.
- No specific roles or authorities are required.
- Authentication is prompted with a simple login page.
- There's only one user; the username is *user*.

This is a good start, but I think that the security needs of most applications (Taco Cloud included) will be quite different from these rudimentary security features.

You have more work to do if you're going to properly secure the Taco Cloud application. You'll need to at least configure Spring Security to do the following:

- Provide a login page that is designed to match the website.
- Provide for multiple users, and enable a registration page so new Taco Cloud customers can sign up.
- Apply different security rules for different request paths. The home page and registration pages, for example, shouldn't require authentication at all.

To meet your security needs for Taco Cloud, you'll have to write some explicit configuration, overriding what autoconfiguration has given you. You'll start by configuring a proper user store so that you can have more than one user.

5.2 Configuring authentication

Over the years, several ways of configuring Spring Security have existed, including lengthy XML configuration. Fortunately, several recent versions of Spring Security have supported Java configuration, which is much easier to read and write.

Before this chapter is finished, you'll have configured all of your Taco Cloud security needs in a Java configuration for Spring Security. But to get started, you'll ease into it by writing the configuration class shown in the following listing.

Listing 5.1 A barebones configuration class for Spring Security

```
package tacos.security;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
```

```
@Configuration
public class SecurityConfig {

  @Bean
  public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
  }

}
```

What does this barebones security configuration do for you? Not much, actually. The main thing it does is declare a `PasswordEncoder` bean, which we'll use both when creating new users and when authenticating users at login. In this case, we're using `BCryptPasswordEncoder`, one of a handful of password encoders provided by Spring Security, including the following:

- `BCryptPasswordEncoder` —Applies bcrypt strong hashing encryption
- `NoOpPasswordEncoder` —Applies no encoding
- `Pbkdf2PasswordEncoder` —Applies PBKDF2 encryption
- `SCryptPasswordEncoder` —Applies Scrypt hashing encryption
- `StandardPasswordEncoder` —Applies SHA-256 hashing encryption

No matter which password encoder you use, it's important to understand that the password in the database is never decoded. Instead, the password that the user enters at login is encoded using the same algorithm, and it's then compared with the encoded password in the database. That comparison is performed in the `PasswordEncoder`'s `matches()` method.

Which password encoder should you use?

Not all password encoders are created equal. Ultimately, you'll need to weigh each password encoder's algorithm against your security goals and decide for yourself. But you should avoid a couple of password encoders for production applications.

`NoOpPasswordEncoder` applies no encryption whatsoever. Therefore, although it may be useful for testing, it is unsuitable for production use. And `StandardPasswordEncoder` is not considered secure enough for password encryption and has, in fact, been deprecated.

Instead, consider one of the other password encoders, any of which are more secure. We're going to use `BCryptPasswordEncoder` for the examples in this book.

In addition to the password encoder, we'll fill in this configuration class with more beans to define the specifics of security for our application. We'll start by configuring a user store that can handle more than one user.

To configure a user store for authentication purposes, you'll need to declare a `UserDetailsService` bean. The `UserDetailsService` interface is relatively simple, including only one method that must be implemented. Here's what `UserDetailsService` looks like:

```
public interface UserDetailsService {

   UserDetails loadUserByUsername(String username) throws UsernameNotFoundException;

}
```

The `loadUserByUsername()` method accepts a username and uses it to look up a `UserDetails` object. If no user can be found for the given username, then it will throw a `UsernameNotFoundException`.

As it turns out, Spring Security offers several out-of-the-box implementations of `UserDetailsService`, including the following:

- An in-memory user store
- A JDBC user store
- An LDAP user store

Or, you can also create your own implementation to suit your application's specific security needs.

To get started, let's try out the in-memory implementation of `UserDetailsService`.

2.1 In-memory user details service

One place where user information can be kept is in memory. Suppose you have only a handful of users, none of which are likely to change. In that case, it may be simple enough to define those users as part of the security configuration.

The following bean method shows how to create an `InMemoryUserDetailsManager` (which implements UserDetailsService) with two users, "buzz" and "woody," for that purpose.

Listing 5.2 Declaring users in an in-memory user details service bean

```
@Bean
public UserDetailsService userDetailsService(PasswordEncoder encoder) {
  List<UserDetails> usersList = new ArrayList<>();
  usersList.add(new User(
     "buzz", encoder.encode("password"),
        Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"))));
  usersList.add(new User(
     "woody", encoder.encode("password"),
        Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"))));
  return new InMemoryUserDetailsManager(usersList);
}
```

Here, a list of Spring Security `User` objects are created, each with a username, password, and a list of one or more authorities. Then an `InMemoryUserDetailsManager` is created using that list.

If you try out the application now, you should be able to log in as either "woody" or "buzz," using *password* as the password.

The in-memory user details service is convenient for testing purposes or for very simple applications, but it doesn't allow for easy editing of users. If you need to add, remove, or change a user, you'll have to make the necessary changes and then rebuild and redeploy the application.

For the Taco Cloud application, you want customers to be able to register with the application and manage their own user accounts. That doesn't fit with the limitations of the in-memory user details service. So let's take a look at how to create our own implementation of `UserDetailsService` that allows for a user store database.

2.2 Customizing user authentication

In the previous chapter, you settled on using Spring Data JPA as your persistence option for all taco, ingredient, and order data. It would thus make sense to persist user data in the same way. If you do so, the data will ultimately reside in a relational database, so

you could use JDBC authentication. But it'd be even better to leverage the Spring Data JPA repository used to store users.

First things first, though. Let's create the domain object and repository interface that represents and persists user information.

DEFINING THE USER DOMAIN AND PERSISTENCE

When Taco Cloud customers register with the application, they'll need to provide more than just a username and password. They'll also give you their full name, address, and phone number. This information can be used for a variety of purposes, including prepopulating the order form (not to mention potential marketing opportunities).

To capture all of that information, you'll create a `User` class, as follows.

Listing 5.3 Defining a user entity

```
package tacos;
import java.util.Arrays;
import java.util.Collection;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.
                                SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import lombok.AccessLevel;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;
@Entity
@Data
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@RequiredArgsConstructor
public class User implements UserDetails {

  private static final long serialVersionUID = 1L;

  @Id
  @GeneratedValue(strategy=GenerationType.AUTO)
  private Long id;
```

```
    private final String username;
    private final String password;
    private final String fullname;
    private final String street;
    private final String city;
    private final String state;
    private final String zip;
    private final String phoneNumber;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
      return Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"));
    }

    @Override
    public boolean isAccountNonExpired() {
      return true;
    }

    @Override
    public boolean isAccountNonLocked() {
      return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
      return true;
    }

    @Override
    public boolean isEnabled() {
      return true;
    }

  }
```

The first thing to notice about this `User` type is that it's not the same as the `User` class we used when creating the in-memory user details service. This one has more details about the user that we'll need to fulfill taco orders, including the user's address and contact information.

You've also probably noticed that the `User` class is a bit more involved than any of the other entities defined in chapter 3. In addition to defining a handful of properties, `User` also implements the `UserDetails` interface from Spring Security.

Implementations of `UserDetails` will provide some essential user information to the framework, such as what authorities are granted to the user and whether the user's account is enabled.

The `getAuthorities()` method should return a collection of authorities granted to the user. The various `is*` methods return a `boolean` to indicate whether the user's account is enabled, locked, or expired.

For your `User` entity, the `getAuthorities()` method simply returns a collection indicating that all users will have been granted `ROLE_USER` authority. And, at least for now, Taco Cloud has no need to disable users, so all of the `is*` methods return `true` to indicate that the users are active.

With the `User` entity defined, you can now define the repository interface as follows:

```
package tacos.data;
import org.springframework.data.repository.CrudRepository;
import tacos.User;

public interface UserRepository extends CrudRepository<User, Long> {

  User findByUsername(String username);

}
```

In addition to the CRUD operations provided by extending `CrudRepository`, `UserRepository` defines a `findByUsername()` method that you'll use in the user details service to look up a `User` by their username.

As you learned in chapter 3, Spring Data JPA automatically generates the implementation of this interface at run time. Therefore, you're now ready to write a custom user details service that uses this repository.

CREATING A USER DETAILS SERVICE

As you'll recall, the `UserDetailsService` interface defines only a single `loadUserByUsername()` method. That means it is a functional interface and can be implemented as a lambda instead of as a full-blown implementation class. Because all we really need is for our custom `UserDetailsService` to delegate to the `UserRepository`, it can be simply declared as a bean using the following configuration method.

Listing 5.4 Defining a custom user details service bean

```
@Bean
public UserDetailsService userDetailsService(UserRepository userRepo) {
  return username -> {
    User user = userRepo.findByUsername(username);
    if (user != null) return user;

    throw new UsernameNotFoundException("User '" + username + "' not found");
  };
}
```

The `userDetailsService()` method is given a `UserRepository` as a parameter. To create the bean, it returns a lambda that takes a `username` parameter and uses it to call `findByUsername()` on the given `UserRepository`.

The `loadByUsername()` method has one simple rule: it must never return `null`. Therefore, if the call to `findByUsername()` returns `null`, the lambda will throw a `UsernameNotFoundException` (which is defined by Spring Security). Otherwise, the `User` that was found will be returned.

Now that you have a custom user details service that reads user information via a JPA repository, you just need a way to get users into the database in the first place. You need to create a registration page for Taco Cloud patrons to register with the application.

REGISTERING USERS

Although Spring Security handles many aspects of security, it really isn't directly involved in the process of user registration, so you're going to rely on a little bit of Spring MVC to handle that task. The `RegistrationController` class in the following listing presents and processes registration forms.

Listing 5.5 A user registration controller

```
package tacos.security;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import tacos.data.UserRepository;
```

```
@Controller
@RequestMapping("/register")
public class RegistrationController {

  private UserRepository userRepo;
  private PasswordEncoder passwordEncoder;

  public RegistrationController(
      UserRepository userRepo, PasswordEncoder passwordEncoder) {
    this.userRepo = userRepo;
    this.passwordEncoder = passwordEncoder;
  }

  @GetMapping
  public String registerForm() {
    return "registration";
  }

  @PostMapping
  public String processRegistration(RegistrationForm form) {
    userRepo.save(form.toUser(passwordEncoder));
    return "redirect:/login";
  }

}
```

Like any typical Spring MVC controller, `RegistrationController` is annotated with `@Controller` to designate it as a controller and to mark it for component scanning. It's also annotated with `@RequestMapping` such that it will handle requests whose path is /register.

More specifically, a `GET` request for /register will be handled by the `registerForm()` method, which simply returns a logical view name of `registration`. The following listing shows a Thymeleaf template that defines the `registration` view.

Listing 5.6 A Thymeleaf registration form view

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
  </head>
```

```
  <body>
    <h1>Register</h1>

    <img th:src="@{/images/TacoCloud.png}"/>

    <form method="POST" th:action="@{/register}" id="registerForm">

      <label for="username">Username: </label>
      <input type="text" name="username"/><br/>

      <label for="password">Password: </label>
      <input type="password" name="password"/><br/>

      <label for="confirm">Confirm password: </label>
      <input type="password" name="confirm"/><br/>

      <label for="fullname">Full name: </label>
      <input type="text" name="fullname"/><br/>

      <label for="street">Street: </label>
      <input type="text" name="street"/><br/>

      <label for="city">City: </label>
      <input type="text" name="city"/><br/>

      <label for="state">State: </label>
      <input type="text" name="state"/><br/>

      <label for="zip">Zip: </label>
      <input type="text" name="zip"/><br/>

      <label for="phone">Phone: </label>
      <input type="text" name="phone"/><br/>

      <input type="submit" value="Register"/>
    </form>
  </body>
</html>
```

When the form is submitted, the `processRegistration()` method handles the HTTPS `POST` request. The form fields will be bound to a `RegistrationForm` object by Spring MVC and passed into the `processRegistration()` method for processing. `RegistrationForm` is defined in the following class:

```
package tacos.security;
import org.springframework.security.crypto.password.PasswordEncoder;
import lombok.Data;
import tacos.User;

@Data
public class RegistrationForm {

  private String username;
  private String password;
  private String fullname;
  private String street;
  private String city;
  private String state;
  private String zip;
  private String phone;

  public User toUser(PasswordEncoder passwordEncoder) {
    return new User(
        username, passwordEncoder.encode(password),
        fullname, street, city, state, zip, phone);
  }

}
```

For the most part, `RegistrationForm` is just a basic Lombok class with a handful of properties. But the `toUser()` method uses those properties to create a new `User` object, which is what `processRegistration()` will save, using the injected `UserRepository`.

You've no doubt noticed that `RegistrationController` is injected with a `PasswordEncoder`. This is the exact same `PasswordEncoder` bean you declared earlier. When processing a form submission, `RegistrationController` passes it to the `toUser()` method, which uses it to encode the password before saving it to the database. In this way, the submitted password is written in an encoded form, and the user details service will be able to authenticate against that encoded password.

Now the Taco Cloud application has complete user registration and authentication support. But if you start it up at this point, you'll notice that you can't even get to the registration page without being prompted to log in. That's because, by default, all requests require authentication. Let's look at how web requests are intercepted and secured so you can fix this strange chicken-and-egg situation.

5.3 Securing web requests

The security requirements for Taco Cloud should require that a user be authenticated before designing tacos or placing orders. But the home page, login page, and registration page should be available to unauthenticated users.

To configure these security rules, we'll need to declare a `SecurityFilterChain` bean. The following `@Bean` method shows a minimal (but not useful) `SecurityFilterChain` bean declaration:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  return http.build();
}
```

The `filterChain()` method accepts an `HttpSecurity` object, which acts as a builder that can be used to configure how security is handled at the web level. Once security configuration is set up via the `HttpSecurity` object, a call to `build()` will create a `SecurityFilterChain` that is returned from the bean method.

The following are among the many things you can configure with `HttpSecurity`:

- Requiring that certain security conditions be met before allowing a request to be served
- Configuring a custom login page
- Enabling users to log out of the application
- Configuring cross-site request forgery protection

Intercepting requests to ensure that the user has proper authority is one of the most common things you'll configure `HttpSecurity` to do. Let's ensure that your Taco Cloud customers meet those requirements.

3.1 Securing requests

You need to ensure that requests for /design and /orders are available only to authenticated users; all other requests should be permitted for all users. The following configuration does exactly that:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

```
    return http
      .authorizeRequests()
        .antMatchers("/design", "/orders").hasRole("USER")
        .antMatchers("/", "/**").permitAll()

      .and()
      .build();
  }
```

The call to `authorizeRequests()` returns an object (`ExpressionUrlAuthorizationConfigurer.ExpressionInterceptUrlRegistry`) on which you can specify URL paths and patterns and the security requirements for those paths. In this case, you specify the following two security rules:

- Requests for /design and /orders should be for users with a granted authority of `ROLE_USER`. Don't include the `ROLE_` prefix on roles passed to `hasRole()`; it will be assumed by `hasRole()`.
- All requests should be permitted to all users.

The order of these rules is important. Security rules declared first take precedence over those declared lower down. If you were to swap the order of those two security rules, all requests would have `permitAll()` applied to them; the rule for /design and /orders requests would have no effect.

The `hasRole()` and `permitAll()` methods are just a couple of the methods for declaring security requirements for request paths. Table 5.1 describes all the available methods.

Table 5.1 Configuration methods to define how a path is to be secured

Method	What it does
access(String)	Allows access if the given Spring Expression Language (SpEL) expression evaluates to true
anonymous()	Allows access to anonymous users
authenticated()	Allows access to authenticated users
denyAll()	Denies access unconditionally
fullyAuthenticated()	Allows access if the user is fully authenticated (not remembered)
hasAnyAuthority(String...)	Allows access if the user has any of the given authorities
hasAnyRole(String...)	Allows access if the user has any of the given roles
hasAuthority(String)	Allows access if the user has the given authority
hasIpAddress(String)	Allows access if the request comes from the given IP address
hasRole(String)	Allows access if the user has the given role
not()	Negates the effect of any of the other access methods
permitAll()	Allows access unconditionally
rememberMe()	Allows access for users who are authenticated via re-member-me

Most of the methods in table 5.1 provide essential security rules for request handling, but they're self-limiting, enabling security rules only as defined by those methods. Alternatively, you can use the `access()` method to provide a SpEL expression to declare

richer security rules. Spring Security extends SpEL to include several security-specific values and functions, as listed in table 5.2.

Table 5.2 Spring Security extensions to the Spring Expression Language

Security expression	What it evaluates to
authentication	The user's authentication object
`denyAll`	Always evaluates to `false`
hasAnyAuthority(String... authorities)	`true` if the user has been granted any of the given authorities
`hasAnyRole(String... roles)`	`true` if the user has any of the given roles
hasAuthority(String authority)	`true` if the user has been granted the specified authority
`hasPermission(Object target, Object permission)`	`true` if the user has access to the specified target object for the given permission
hasPermission(Serializable targetId, String targetType, Object permission)	`true` if the user has access to the object specified by `targetId` and the specified `targetType` for the given permission
`hasRole(String role)`	`true` if the user has the given role
hasIpAddress(String ipAddress)	`true` if the request comes from the given IP address
`isAnonymous()`	`true` if the user is anonymous
isAuthenticated()	`true` if the user is authenticated
`isFullyAuthenticated()`	`true` if the user is fully authenticated (not authenticated with `remember-me`)

isRememberMe()	true if the user is authenticated via remember-me
permitAll	Always evaluates to true
principal	The user's principal object

As you can see, most of the security expression extensions in table 5.2 correspond to similar methods in table 5.1. In fact, using the `access()` method along with the `hasRole()` and `permitAll` expressions, you can rewrite the `SecurityFilterChain` configuration as follows.

Listing 5.7 Using Spring expressions to define authorization rules

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  return http
    .authorizeRequests()
      .antMatchers("/design", "/orders").access("hasRole('USER')")
      .antMatchers("/", "/**").access("permitAll()")

    .and()
    .build();
}
```

This may not seem like a big deal at first. After all, these expressions only mirror what you already did with method calls. But expressions can be much more flexible. For instance, suppose that (for some crazy reason) you wanted to allow only users with `ROLE_USER` authority to create new tacos on Tuesdays (for example, on Taco Tuesday); you could rewrite the expression as shown in this modified version of the `SecurityFilterChain` bean method:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  return http
    .authorizeRequests()
      .antMatchers("/design", "/orders")
        .access("hasRole('USER') && " +
          "T(java.util.Calendar).getInstance().get("+
          "T(java.util.Calendar).DAY_OF_WEEK) == " +
```

```
        "T(java.util.Calendar).TUESDAY")
      .antMatchers("/", "/**").access("permitAll()")

    .and()
    .build();
  }
```

With SpEL security constraints, the possibilities are virtually endless. I'll bet that you're already dreaming up interesting security constraints based on SpEL.

The authorization needs for the Taco Cloud application are met by the simple use of `access()` and the SpEL expressions. Now let's see about customizing the login page to fit the look of the Taco Cloud application.

3.2 Creating a custom login page

The default login page is much better than the clunky HTTP basic dialog box you started with, but it's still rather plain and doesn't quite fit with the look of the rest of the Taco Cloud application.

To replace the built-in login page, you first need to tell Spring Security what path your custom login page will be at. That can be done by calling `formLogin()` on the `HttpSecurity` object, as shown next:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  return http
    .authorizeRequests()
      .antMatchers("/design", "/orders").access("hasRole('USER')")
      .antMatchers("/", "/**").access("permitAll()")

    .and()
      .formLogin()
        .loginPage("/login")

    .and()
    .build();
  }
```

Notice that before you call `formLogin()`, you bridge this section of configuration and the previous section with a call to `and()`. The `and()` method signifies that you're finished

with the authorization configuration and are ready to apply some additional HTTP configuration. You'll use `and()` several times as you begin new sections of configuration.

After the bridge, you call `formLogin()` to start configuring your custom login form. The call to `loginPage()` after that designates the path where your custom login page will be provided. When Spring Security determines that the user is unauthenticated and needs to log in, it will redirect them to this path.

Now you need to provide a controller that handles requests at that path. Because your login page will be fairly simple—nothing but a view—it's easy enough to declare it as a view controller in `WebConfig`. The following `addViewControllers()` method sets up the login page view controller alongside the view controller that maps "/" to the home controller:

```
@Override
public void addViewControllers(ViewControllerRegistry registry) {
  registry.addViewController("/").setViewName("home");
  registry.addViewController("/login");
}
```

Finally, you need to define the login page view itself. Because you're using Thymeleaf as your template engine, the following Thymeleaf template should do fine:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
  </head>

  <body>
    <h1>Login</h1>
    <img th:src="@{/images/TacoCloud.png}"/>

    <div th:if="${error}">
      Unable to login. Check your username and password.
    </div>

    <p>New here? Click
      <a th:href="@{/register}">here</a> to register.</p>

    <form method="POST" th:action="@{/login}" id="loginForm">
```

```
      <label for="username">Username: </label>
      <input type="text" name="username" id="username" /><br/>

      <label for="password">Password: </label>
      <input type="password" name="password" id="password" /><br/>

      <input type="submit" value="Login"/>
    </form>
  </body>
</html>
```

The key things to note about this login page are the path it posts to and the names of the username and password fields. By default, Spring Security listens for login requests at /login and expects that the username and password fields be named `username` and `password`. This is configurable, however. For example, the following configuration customizes the path and field names:

```
.and()
  .formLogin()
    .loginPage("/login")
    .loginProcessingUrl("/authenticate")
    .usernameParameter("user")
    .passwordParameter("pwd")
```

Here, you specify that Spring Security should listen for requests to /authenticate to handle login submissions. Also, the username and password fields should now be named `user` and `pwd`.

By default, a successful login will take the user directly to the page that they were navigating to when Spring Security determined that they needed to log in. If the user were to directly navigate to the login page, a successful login would take them to the root path (for example, the home page). But you can change that by specifying a default success page, as shown next:

```
.and()
  .formLogin()
    .loginPage("/login")
    .defaultSuccessUrl("/design")
```

As configured here, if the user were to successfully log in after directly going to the login page, they would be directed to the /design page.

Optionally, you can force the user to the design page after login, even if they were navigating elsewhere prior to logging in, by passing `true` as a second parameter to `defaultSuccessUrl` as follows:

```
.and()
  .formLogin()
    .loginPage("/login")
    .defaultSuccessUrl("/design", true)
```

Signing in with a username and password is the most common way to authenticate in a web application. But let's have a look at another way to authenticate a user that uses someone else's login page.

3.3 Enabling third-party authentication

You may have seen links or buttons on your favorite website that say "Sign in with Facebook," "Log in with Twitter," or something similar. Rather than asking a user to enter their credentials on a login page specific to the website, they offer a way to sign in via another website like Facebook that they may already be logged into.

This type of authentication is based on OAuth2 or OpenID Connect (OIDC). Although OAuth2 is an authorization specification, and we'll talk more about how to use it to secure REST APIs in chapter 8, it can be also used to perform authentication via a third-party website. OpenID Connect is another security specification that is based on OAuth2 to formalize the interaction that takes place during a third-party authentication.

To employ this type of authentication in your Spring application, you'll need to add the OAuth2 client starter to the build as follows:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Then, at the very least, you'll need to configure details about one or more OAuth2 or OpenID Connect servers that you want to be able to authenticate against. Spring

Security supports sign-in with Facebook, Google, GitHub, and Okta out of the box, but you can configure other clients by specifying a few extra properties.

The general set of properties you'll need to set for your application to act as an OAuth2/OpenID Connect client follows:

```
spring:
  security:
    oauth2:
      client:
        registration:
          <oauth2 or openid provider name>:
            clientId: <client id>
            clientSecret: <client secret>
            scope: <comma-separated list of requested scopes>
```

For example, suppose that for Taco Cloud, we want users to be able to sign in using Facebook. The following configuration in application.yml will set up the OAuth2 client:

```
spring:
  security:
    oauth2:
      client:
        registration:
          facebook:
            clientId: <facebook client id>
            clientSecret: <facebook client secret>
            scope: email, public_profile
```

The client ID and secret are the credentials that identify your application to Facebook. You can obtain a client ID and secret by creating a new application entry at **https://developers.facebook.com/**. The `scope` property specifies the access that the application will be granted. In this case, the application will have access to the user's email address and the essential information from their public Facebook profile.

In a very simple application, this is all you will need. When the user attempts to access a page that requires authentication, their browser will redirect to Facebook. If they're not already logged in to Facebook, they'll be greeted with the Facebook sign-in page. After signing in to Facebook, they'll be asked to authorize your application and grant the requested scope. Finally, they'll be redirected back to your application, where they will have been authenticated.

If, however, you've customized security by declaring a `SecurityFilterChain` bean, then you'll need to enable OAuth2 login along with the rest of the security configuration as follows:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  return http
    .authorizeRequests()
      .mvcMatchers("/design", "/orders").hasRole("USER")
      .anyRequest().permitAll()

    .and()
      .formLogin()
        .loginPage("/login")

    .and()
      .oauth2Login()

  ...

    .and()
    .build();
}
```

You may also want to offer both a traditional username-password login and third-party login. In that case, you can specify the login page in the configuration like this:

```
.and()
  .oauth2Login()
    .loginPage("/login")
```

This will cause the application to always take the user to the application-provided login page where they may choose to log in with their username and password as usual. But you can also provide a link on that same login page that offers them the opportunity to log in with Facebook. Such a link could look like this in the login page's HTML template:

```
<a th:href="@{/oauth2/authorization/facebook}">Sign in with Facebook</a>
```

Now that you've dealt with logging in, let's flip to the other side of the authentication coin and see how you can enable a user to log out. Just as important as logging in to an

application is logging out. To enable logout, you simply need to call `logout` on the `HttpSecurity` object as follows:

```
.and()
  .logout()
```

This sets up a security filter that intercepts `POST` requests to /logout. Therefore, to provide logout capability, you just need to add a logout form and button to the views in your application, as shown next:

```
<form method="POST" th:action="@{/logout}">
  <input type="submit" value="Logout"/>
</form>
```

When the user clicks the button, their session will be cleared, and they will be logged out of the application. By default, they'll be redirected to the login page where they can log in again. But if you'd rather they be sent to a different page, you can call `logoutSuccessUrl()` to specify a different post-logout landing page, as shown here:

```
.and()
  .logout()
    .logoutSuccessUrl("/")
```

In this case, users will be sent to the home page following logout.

3.4 Preventing cross-site request forgery

Cross-site request forgery (CSRF) is a common security attack. It involves subjecting a user to code on a maliciously designed web page that automatically (and usually secretly) submits a form to another application on behalf of a user who is often the victim of the attack. For example, a user may be presented with a form on an attacker's website that automatically posts to a URL on the user's banking website (which is presumably poorly designed and vulnerable to such an attack) to transfer money. The user may not even know that the attack happened until they notice money missing from their account.

To protect against such attacks, applications can generate a CSRF token upon displaying a form, place that token in a hidden field, and then stow it for later use on the server. When the form is submitted, the token is sent back to the server along with the rest of the form data. The request is then intercepted by the server and compared with the to-

ken that was originally generated. If the token matches, the request is allowed to proceed. Otherwise, the form must have been rendered by an evil website without knowledge of the token generated by the server.

Fortunately, Spring Security has built-in CSRF protection. Even more fortunate is that it's enabled by default and you don't need to explicitly configure it. You only need to make sure that any forms your application submits include a field named `_csrf` that contains the CSRF token.

Spring Security even makes that easy by placing the CSRF token in a request attribute with the name `_csrf`. Therefore, you could render the CSRF token in a hidden field with the following in a Thymeleaf template:

```
<input type="hidden" name="_csrf" th:value="${_csrf.token}"/>
```

If you're using Spring MVC's JSP tag library or Thymeleaf with the Spring Security dialect, you needn't even bother explicitly including a hidden field. The hidden field will be rendered automatically for you.

In Thymeleaf, you just need to make sure that one of the attributes of the `<form>` element is prefixed as a Thymeleaf attribute. That's usually not a concern, because it's quite common to let Thymeleaf render the path as context relative. For example, the `th:action` attribute shown next is all you need for Thymeleaf to render the hidden field for you:

```
<form method="POST" th:action="@{/login}" id="loginForm">
```

It's possible to disable CSRF support, but I'm hesitant to show you how. CSRF protection is important and easily handled in forms, so there's little reason to disable it. But if you insist on disabling it, you can do so by calling `disable()` like this:

```
.and()
  .csrf()
    .disable()
```

Again, I caution you not to disable CSRF protection, especially for production applications.

All of your web layer security is now configured for Taco Cloud. Among other things, you now have a custom login page and the ability to authenticate users against a JPA user repository. Now let's see how you can obtain information about the logged-in user.

5.4 Applying method-level security

Although it's easy to think about security at the web-request level, that's not always where security constraints are best applied. Sometimes it's better to verify that the user is authenticated and has been granted adequate authority at the point where the secured action will be performed.

For example, let's say that for administrative purposes, we have a service class that includes a method for clearing out all orders from the database. Using an injected `OrderRepository`, that method might look something like this:

```
public void deleteAllOrders() {
  orderRepository.deleteAll();
}
```

Now, suppose we have a controller that calls the `deleteAllOrders()` method as the result of a `POST` request, as shown here:

```
@Controller
@RequestMapping("/admin")
public class AdminController {

  private OrderAdminService adminService;

  public AdminController(OrderAdminService adminService) {
    this.adminService = adminService;
  }

  @PostMapping("/deleteOrders")
  public String deleteAllOrders() {
    adminService.deleteAllOrders();
    return "redirect:/admin";
  }

}
```

It'd be easy enough to tweak `SecurityConfig` as follows to ensure that only authorized users are allowed to perform that `POST` request:

```
.authorizeRequests()
  ...
  .antMatchers(HttpMethod.POST, "/admin/**")
        .access("hasRole('ADMIN')")
  ....
```

That's great and would prevent any unauthorized user from making a `POST` request to /admin/deleteOrders that would result in all orders disappearing from the database.

But suppose that some other controller method also calls `deleteAllOrders()`. You'd need to add more matchers to secure the requests for the other controllers that will need to be secured.

Instead, we can apply security directly on the `deleteAllOrders()` method like this:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteAllOrders() {
  orderRepository.deleteAll();
}
```

The `@PreAuthorize` annotation takes a SpEL expression, and, if the expression evaluates to `false`, the method will not be invoked. On the other hand, if the expression evaluates to `true`, then the method will be allowed. In this case, `@PreAuthorize` is checking that the user has the `ROLE_ADMIN` privilege. If so, then the method will be called and all orders will be deleted. Otherwise, it will be stopped in its tracks.

In the event that `@PreAuthorize` blocks the call, then Spring Security's `AccessDeniedException` will be thrown. This is an unchecked exception, so you don't need to catch it, unless you want to apply some custom behavior around the exception handling. If left uncaught, it will bubble up and eventually be caught by Spring Security's filters and handled accordingly, either with an HTTP 403 page or perhaps by redirecting to the login page if the user is unauthenticated.

For `@PreAuthorize` to work, you'll need to enable global method security. For that, you'll need to annotate the security configuration class with `@EnableGlobalMethodSecurity` as follows:

```
@Configuration
@EnableGlobalMethodSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
  ...
}
```

You'll find `@PreAuthorize` to be a useful annotation for most method-level security needs. But know that it has a slightly less useful after-invocation counterpart in `@PostAuthorize`. The `@PostAuthorize` annotation works almost the same as the `@PreAuthorize` annotation, except that its expression won't be evaluated until after the target method is invoked and returns. This allows the expression to consider the return value of the method in deciding whether to permit the method invocation.

For example, suppose we have a method that fetches an order by its ID. If you want to restrict it from being used except by admins or by the user who the order belongs to, you can use `@PostAuthorize` like this:

```
@PostAuthorize("hasRole('ADMIN') || " +
    "returnObject.user.username == authentication.name")
public TacoOrder getOrder(long id) {
  ...
}
```

In this case, the `returnObject` is the `TacoOrder` returned from the method. If its `user` property has a `username` that is equal to the authentication's `name` property, then it will be allowed. To know that, though, the method will need to be executed so that it can return the `TacoOrder` object for consideration.

But wait! How can you secure a method from being invoked if the condition for applying security relies on the return value from the method invocation? That chicken-and-egg riddle is solved by allowing the method to be invoked, then throwing an `AccessDeniedException` if the expression returns `false`.

5.5 Knowing your user

Often, it's not enough to simply know that the user has logged in and what permissions they have been granted. It's usually important to also know who they are, so that you can tailor their experience.

For example, in `OrderController`, when you initially create the `TacoOrder` object that's bound to the order form, it'd be nice if you could prepopulate the `TacoOrder` with the user's name and address, so they don't have to reenter it for each order. Perhaps even more important, when you save their order, you should associate the `TacoOrder` entity with the `User` that created the order.

To achieve the desired connection between an `TacoOrder` entity and a `User` entity, you need to add the following new property to the `TacoOrder` class:

```
@Data
@Entity
@Table(name="Taco_Order")
public class TacoOrder implements Serializable {

  ...

  @ManyToOne
  private User user;

  ...

}
```

The `@ManyToOne` annotation on this property indicates that an order belongs to a single user and, conversely, that a user may have many orders. (Because you're using Lombok, you won't need to explicitly define accessor methods for the property.)

In `OrderController`, the `processOrder()` method is responsible for saving an order. It will need to be modified to determine who the authenticated user is and to call `setUser()` on the `TacoOrder` object to connect the order with the user.

We have several ways to determine who the user is. A few of the most common ways follow:

- Inject a `java.security.Principal` object into the controller method.
- Inject an `org.springframework.security.core.Authentication` object into the controller method.
- Use `org.springframework.security.core.context.SecurityContextHolder` to get at the security context.

- Inject an `@AuthenticationPrincipal` annotated method parameter. (`@AuthenticationPrincipal` is from Spring Security's `org.springframework.security.core.annotation` package.)

For example, you could modify `processOrder()` to accept a `java.security.Principal` as a parameter. You could then use the principal name to look up the user from a `UserRepository` as follows:

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    Principal principal) {

  ...

  User user = userRepository.findByUsername(
        principal.getName());

  order.setUser(user);

  ...

}
```

This works fine, but it litters code that's otherwise unrelated to security with security code. You can trim down some of the security-specific code by modifying `processOrder()` to accept an `Authentication` object as a parameter instead of a `Principal`, as shown next:

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    Authentication authentication) {

  ...

  User user = (User) authentication.getPrincipal();
  order.setUser(user);

  ...

}
```

With the `Authentication` in hand, you can call `getPrincipal()` to get the principal object which, in this case, is a `User`. Note that `getPrincipal()` returns a `java.util` `.Object`, so you need to cast it to `User`.

Perhaps the cleanest solution of all, however, is to simply accept a `User` object in `processOrder()` but annotate it with `@AuthenticationPrincipal` so that it will be the authentication's principal, as follows:

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    @AuthenticationPrincipal User user) {

  if (errors.hasErrors()) {
    return "orderForm";
  }

  order.setUser(user);

  orderRepo.save(order);
  sessionStatus.setComplete();

  return "redirect:/";
}
```

What's nice about `@AuthenticationPrincipal` is that it doesn't require a cast (as with `Authentication`), and it limits the security-specific code to the annotation itself. By the time you get the `User` object in `processOrder()`, it's ready to be used and assigned to the `TacoOrder`.

There's one other way of identifying who the authenticated user is, although it's a bit messy in the sense that it's very heavy with security-specific code. You can obtain an `Authentication` object from the security context and then request its principal like this:

```
Authentication authentication =
    SecurityContextHolder.getContext().getAuthentication();
User user = (User) authentication.getPrincipal();
```

Although this snippet is thick with security-specific code, it has one advantage over the other approaches described: it can be used anywhere in the application, not just in a controller's handler methods. This makes it suitable for use in lower levels of the code.

Summary

- Spring Security autoconfiguration is a great way to get started with security, but most applications will need to explicitly configure security to meet their unique security requirements.
- User details can be managed in user stores backed by relational databases, LDAP, or completely custom implementations.
- Spring Security automatically protects against CSRF attacks.
- Information about the authenticated user can be obtained via the `SecurityContext` object (returned from `SecurityContextHolder.getContext()`) or injected into controllers using `@AuthenticationPrincipal`.

Previous chapter

< 4 Working with nonrelational data

Next chapter

6 Working with configuration properties >